



Опрос про ИИ внутри разработки

[Войти](#)

kmoseenk

10 минут назад

Apple выпустила Mac с багом в ПЗУ. 35 лет его спасала некорректно закодированная инструкция 68030

 Сложный 20 мин 254

Блог компании OTUS, Реверс-инжиниринг*, Старое железо, Компьютерное железо

[Ретроспектива](#)

Это история о том, как Apple допустила ошибку в ПЗУ Macintosh Classic II. Она, скорее всего, должна была помешать ему загрузиться, но каким-то чудом процессор Motorola MC68030 вместо этого спас ситуацию: случайно предотвратил падение системы, выполнив инструкцию с неопределённым поведением.

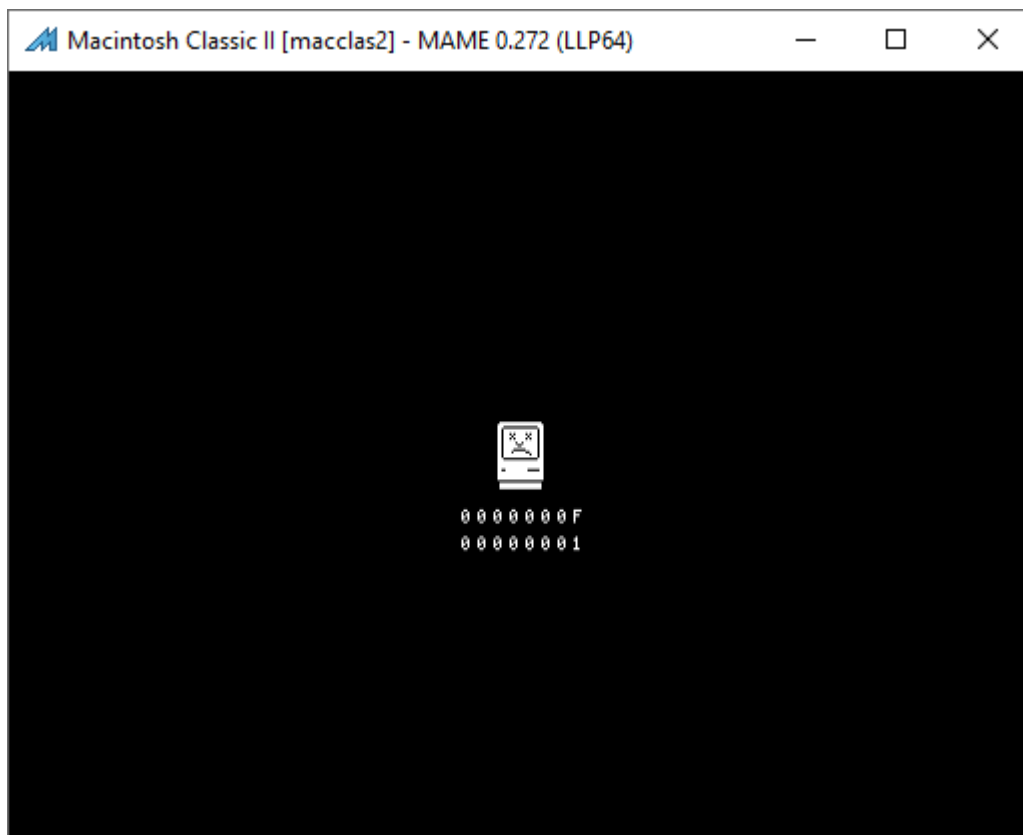
► [TL;DR:](#)

В последнее время я много ковыряюсь в [MAME](#). Если вы о нём не слышали, это эмулятор, прежде всего известный поддержкой множества аркадных игр. Но этим его возможности далеко не ограничиваются. Во многом благодаря невероятной работе [Arbee MAME](#), пожалуй, стал самым полным эмулятором моделей Mac на базе семейства 68000. Чаще всего использую MAME именно для эмуляции Mac.

Вот с чего началось это приключение. Я исправлял в MAME проблемы с сочетанием клавиш Command + Power, которое вызывает отладчик, и решил проверить, работает ли оно на Classic II. У этой модели Mac есть физическая кнопка прерывания сбоку, но также в ней установлен микроконтроллер «Egret» 68HC05, который, помимо прочего, обслуживает клавиатуру и мышь. Он должен уметь распознавать это сочетание и посылать главному процессору немаскируемое прерывание. Насколько я понимаю, по

умолчанию Egret отключает это сочетание, но в [MacsBug](#) есть код, который отправляет команду на его включение.

Проверка сочетания Command + Power в эмулируемом MAME Classic II далеко меня не увела: я заметил нечто очень странное. В режиме 24-битной адресации он совершенно нормально загружался, но при включении 32-битной адресации, необходимой для загрузки MacsBug, не загружался вовсе. Сразу появлялся экран ошибки Sad Mac со звуковым сигналом [Chimes of Death](#). На этой модели «предсмертный звон» состоит из нескольких нот темы из «Сумеречной зоны».



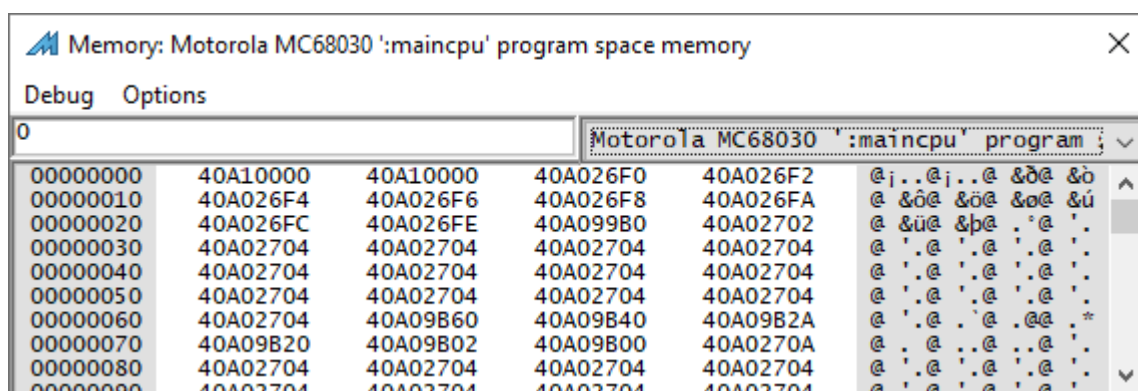
Если вы не знакомы с историей перехода Apple от 24- к 32-битной адресации, коротко перескажу её. Исходный Motorola 68000 имел только 24 адресные линии, хотя внутри работал с 32-битными адресами. Apple приспособила эти восемь неиспользуемых старших бит для хранения флагов: так сэкономили дефицитную ОЗУ. Когда появились новые машины и процессоры с полноценным 32-битным адресным пространством, использовать старший байт для флагов стало нельзя. Из-за этого старое ПО оказалось бы несовместимо с новыми системами, поэтому в них добавили два режима: 24-битный — для совместимости со старым софтом, 32-битный — чтобы задействовать всю установленную ОЗУ.

Так почему же Classic II не загружался в MAME в 32-битном режиме? Что пошло не так? Arbee тоже воспроизвёл проблему, так что по крайней мере я убедился, что не схожу с ума. Я решил, что это какой-то случайный баг MAME, и начал копать глубже, чтобы понять, что именно нужно исправить.

Согласно старой статье из Apple Tech Info Library, код 0000000F означает, что произошло исключение, а 00000001 — что этим исключением стала ошибка шины. На Mac семейства 68k ошибка шины обычно означает попытку обратиться к недопустимому адресу — например, прочитать данные с карты расширения или записать на неё что-то, когда никакой карты в слот не установлено.

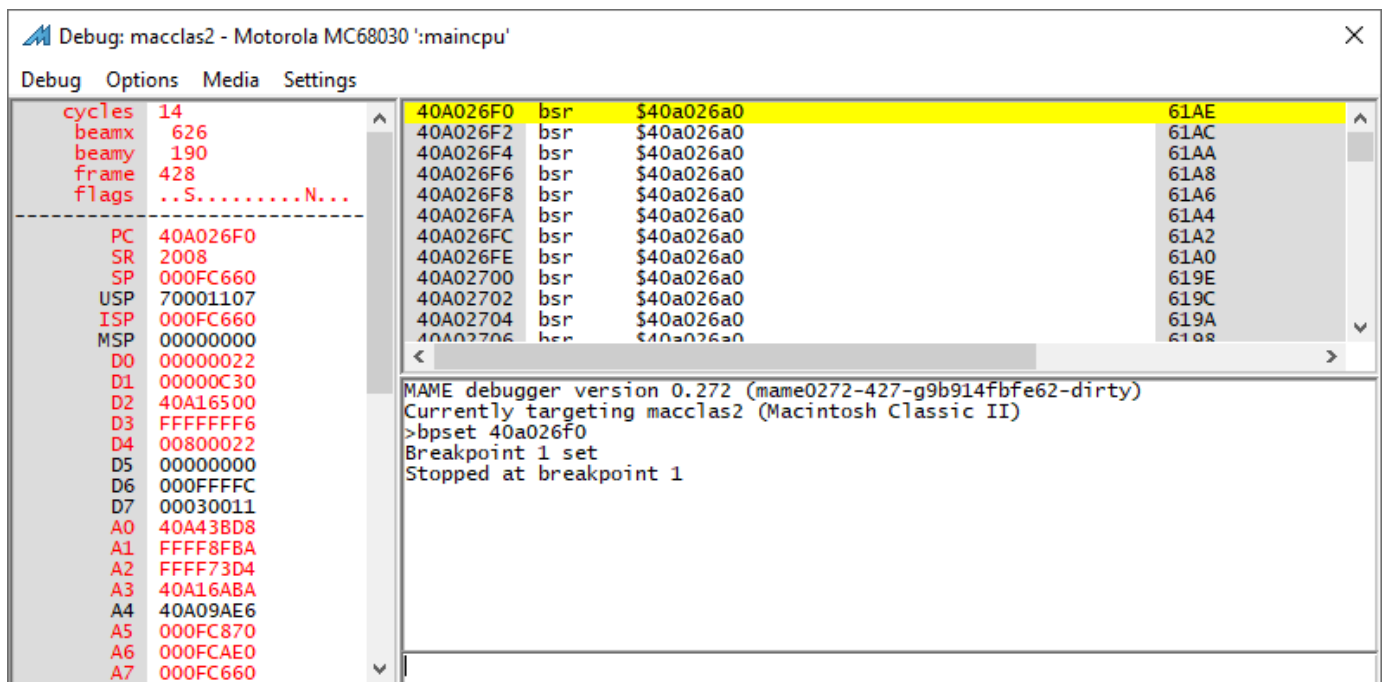
Но к какому именно недопустимому адресу шло обращение? Чтобы понять, что привело к падению, я решил пошагово пройти код в отличном отладчике MAME. Разобраться, что происходит в ПЗУ без какого-либо контекста, бывает непросто. К счастью, Apple включала карты символов для ряда ПЗУ Mac в Macintosh Programmer's Workshop (MPW) — своей среде разработки.

Идти назад от самого экрана Sad Mac было бы неудобно: за его подготовкой и выводом стоит огромный объём кода. Поэтому я решил упростить себе задачу: поставить точку останова на обработчик ошибки шины, а затем уже оттуда проследить выполнение назад. Таблица векторов исключений 68030 начинается в самом начале адресного пространства, а вектор ошибки шины находится по адресу 0x00000008. Пока на экране ещё отображалась ошибка Sad Mac, память по этому адресу выглядела так:



Address	Value	Value	Value	Value	Value
00000000	40A10000	40A10000	40A026F0	40A026F2	@j..@j..@ &ð@ &ð
00000010	40A026F4	40A026F6	40A026F8	40A026FA	@ &ô@ &ô@ &ô@ &ú
00000020	40A026FC	40A026FE	40A099B0	40A02702	@ &ü@ &þ@ .@ .@
00000030	40A02704	40A02704	40A02704	40A02704	@ .@ .@ .@ .@ .@
00000040	40A02704	40A02704	40A02704	40A02704	@ .@ .@ .@ .@ .@
00000050	40A02704	40A02704	40A02704	40A02704	@ .@ .@ .@ .@ .@
00000060	40A02704	40A09B60	40A09B40	40A09B2A	@ .@ .@ .@ .@ .@
00000070	40A09B20	40A09B02	40A09B00	40A0270A	@ .@ .@ .@ .@ .@
00000080	40A02704	40A02704	40A02704	40A02704	@ .@ .@ .@ .@ .@
00000090	40A02704	40A02704	40A02704	40A02704	@ .@ .@ .@ .@ .@

Это означало, что обработчик ошибки шины расположен по адресу 0x40A026F0. В карте символов ПЗУ он также обозначен как GenExcrps. Я выполнил аппаратный сброс эмулируемой машины, поставил точку останова по этому адресу и дождался её срабатывания. Похоже, GenExcrps представляет собой большой список инструкций BSR, которые все переходят на 0x40A026A0 — общий код обработки ошибок, обозначенный в карте символов ПЗУ как ToDeepShit. Отличное название, Apple!



Так или иначе, MAME остановился на моей точке останова, а значит, техническая заметка Apple была права: это действительно оказалась ошибка шины. С помощью команды history в отладчике MAME я вывел обратную трассу инструкций, которые привели к этой точке. Конец вывода history показан в нижней панели на скриншоте ниже:

Если двигаться по трассе вверх, видно, что инструкция, вызвавшая ошибку шины, находится по адресу 0x40A43B9C:

```
move.b #$90, ($1c00,A1)
```

Я открыл этот участок в IDA — с кодом Mac на 68k я до сих пор работаю именно в ней, просто по привычке. Судя по всему, это часть подпрограммы, начинающейся по адресу

0x40A43B40; карта символов ПЗУ услужливо называет её InstallSoundIntHandler. Давайте разберём всю функцию подробнее.

```
ROM:40A43B3E                align $10
ROM:40A43B40
ROM:40A43B40 InstallSoundIntHandler:           ; CODE XREF: ROM:40A43B46↓j
ROM:40A43B40                                     ; DATA XREF: ROM:InstallSoundIntHandler↓o
ROM:40A43B40                lea        ((V8SndIntPatch1-InstallSoundIntHandler)).l,a2
ROM:40A43B46                jmp        InstallSoundIntHandler(pc,a2.l)
ROM:40A43B4A ; -----
```

Первым делом она сразу переходит к V8SndIntPatch1. По-видимому, это патч, добавленный в ПЗУ для инициализации звука в V8. Для контекста: Classic II работает не на восьмицилиндровом бензиновом двигателе. V8 — это название специализированной микросхемы, которую Apple впервые использовала в Macintosh LC. Вот что говорится в аппаратной документации для LC:

- A new custom VLSI (very large scale integration) chip, the V8 gate array, is the heart of the hardware design. It integrates the timing, address decode, video generation, clock generation, sound control, and GLU (general logic unit) functions that were provided by individual chips in earlier Macintosh computers.

Новая специализированная **БИС** сверхвысокой степени интеграции — вентиляная матрица V8 — стала центральным элементом аппаратной архитектуры компьютера. Она объединяет функции синхронизации, дешифрации адресов, формирования видеосигнала, генерации тактовых сигналов, управления звуком и **GLU**, которые в более ранних компьютерах Macintosh были реализованы отдельными микросхемами.

Почему вообще здесь всплыл LC? Потому что с точки зрения архитектуры Classic II гораздо ближе к LC, чем к исходному Macintosh Classic. Вот как это объясняется в соответствующей документации для Classic II:

Although the physical appearance of the Macintosh Classic II computer is very similar to that of the Macintosh Classic, the electrical design of the Macintosh Classic II is based as much as possible on the Macintosh LC architecture. Figure 1-1 is an overall block diagram of the Macintosh Classic II. Notice that the number of components has been significantly reduced through the use of custom integrated circuits. The following are the major changes in the Macintosh Classic II design:

- The Macintosh Classic II uses an MC68030 processor rather than the MC68020 used by the Macintosh LC.
- A new custom VLSI (very large-scale integration) chip, the EAGLE gate array, is the heart of the hardware design. It integrates the timing, memory mapping, video generation, clock generation, sound control, and glue functions that were provided by individual chips in the earlier Macintosh computers.

► [перевод текста изображения](#)

Текстовое описание вентиляционной матрицы EAGLE очень похоже на описание V8, так что неудивительно, что и сами микросхемы во многом похожи. В MAME они обе обрабатываются одним исходным файлом. К чему я веду: вполне логично, что в ПЗУ Classic II встречается код со ссылками на V8. С этим разобрались — теперь посмотрим на V8SndIntPatch1:

```
ROM:40A4C766 V8SndIntPatch1:                                ; DATA XREF: ROM:InstallSoundIntHandlerfo
ROM:40A4C766          move.l  #'hdwr',d0
ROM:40A4C76C          _Gestalt
ROM:40A4C76E          move.l  a0,d0
ROM:40A4C770          btst    #3,d0
ROM:40A4C774          beq.s    locret_40A4C780
ROM:40A4C776          loc_40A4C776:                          ; CODE XREF: ROM:40A4C77C↓j
ROM:40A4C776          ; DATA XREF: ROM:loc_40A4C776↓o
ROM:40A4C776          lea      ((V8SndIntPatch1Rtn-loc_40A4C776)).l,a2
ROM:40A4C77C          jmp      loc_40A4C776(pc,a2.l)
ROM:40A4C780 ; -----
ROM:40A4C780          locret_40A4C780:                        ; CODE XREF: ROM:40A4C774↑j
ROM:40A4C780          rts
ROM:40A4C780 ; -----
```

Этот фрагмент кода вызывает системный трап Gestalt, через который можно получить разную информацию о Mac. В частности, здесь используется селектор gestaltHardwareAttr, определённый в публичных заголовочных файлах Apple как hdwr.

Если в ответе не установлен бит 3 (gestaltHasASC), код сразу выходит из подпрограммы. В противном случае он переходит к V8SndIntPatch1Rtn по адресу 0x40A43B4A — этот адрес можно было увидеть в трассе выполнения на предыдущем скриншоте из отладчика

MAME. Я довольно глубоко разобрался с аппаратными таблицами Classic II и могу подтвердить: gestaltHasASC на Classic II точно установлен. В конце концов, EAGLE содержит урезанный аналог Apple Sound Chip (ASC).

Теперь посмотрим на V8SndIntPatch1Rtn:


```

ROM:40A43B4A U8SndIntPatch1Rtn: ; DATA XREF: ROM:loc_40A4C776↓o
ROM:40A43B4A         moveq    #0,d0
ROM:40A43B4C         move.b   (byte_C83).w,d0
ROM:40A43B50         bpl.s    loc_40A43B54
ROM:40A43B52         rts
ROM:40A43B54         ; -----
ROM:40A43B54         loc_40A43B54: ; CODE XREF: ROM:40A43B50↑j
ROM:40A43B54         movea.l   (dword_2B6).w,a0
ROM:40A43B58         movea.l   $110(a0),a0
ROM:40A43B5C         clr.l     $22(a0)
ROM:40A43B60         clr.l     $1E(a0)
ROM:40A43B64         lea       loc_40A43BD8,a0
ROM:40A43B68         move.l    a0,(dword_D80).w
ROM:40A43B6C         add.w     d0,d0
ROM:40A43B6E         jmp      loc_40A43B72(pc,d0.w)
ROM:40A43B72         ; -----
ROM:40A43B72         loc_40A43B72: ; CODE XREF: ROM:40A43B6E↑j
ROM:40A43B72         bra.s     loc_40A43B92
ROM:40A43B74         ; -----
ROM:40A43B74         bra.s     loc_40A43B92
ROM:40A43B76         ; -----
ROM:40A43B76         bra.s     loc_40A43B92
ROM:40A43B78         ; -----
ROM:40A43B78         bra.s     loc_40A43B92
ROM:40A43B7A         ; -----
ROM:40A43B7A         bra.s     locret_40A43BA4
ROM:40A43B7C         ; -----
ROM:40A43B7C         bra.s     loc_40A43BA6
ROM:40A43B7E         ; -----
ROM:40A43B7E         bra.s     loc_40A43B92
ROM:40A43B80         ; -----
ROM:40A43B80         bra.s     loc_40A43BB2
ROM:40A43B82         ; -----
ROM:40A43B82         bra.s     loc_40A43BA6
ROM:40A43B84         ; -----
ROM:40A43B84         bra.s     loc_40A43BA6
ROM:40A43B86         ; -----
ROM:40A43B86         bra.s     loc_40A43BA6
ROM:40A43B88         ; -----
ROM:40A43B88         bra.s     loc_40A43BA6
ROM:40A43B8A         ; -----
ROM:40A43B8A         bra.s     loc_40A43BA6
ROM:40A43B8C         ; -----
ROM:40A43B8C         bra.s     locret_40A43BA4
ROM:40A43B8E         ; -----
ROM:40A43B8E         bra.s     loc_40A43BA6
ROM:40A43B90         ; -----
ROM:40A43B90         bra.s     loc_40A43BA6
ROM:40A43B92         ; -----
ROM:40A43B92         loc_40A43B92: ; CODE XREF: ROM:loc_40A43B72↑j
ROM:40A43B92         ; ROM:40A43B74↑j ...
ROM:40A43B92         movea.l   (dword_CEC).w,a1
ROM:40A43B96         bclr      #4,$1800(a1)
ROM:40A43B9C         move.b    #$90,$1C00(a1)
ROM:40A43BA2         rts
ROM:40A43BA4         ; -----
ROM:40A43BA4         locret_40A43BA4: ; CODE XREF: ROM:40A43B7A↑j
ROM:40A43BA4         ; ROM:40A43B8C↑j
ROM:40A43BA4         rts
ROM:40A43BA6         ; -----
ROM:40A43BA6         loc_40A43BA6: ; CODE XREF: ROM:40A43B7C↑j
ROM:40A43BA6         ; ROM:40A43B82↑j ...
ROM:40A43BA6         movea.l   (dword_CEC).w,a0
ROM:40A43BAA         move.b    #$90,$13(a0)
ROM:40A43BB0         rts
ROM:40A43BB2         ; -----
ROM:40A43BB2         loc_40A43BB2: ; CODE XREF: ROM:40A43B80↑j
ROM:40A43BB2         movea.l   (dword_CEC).w,a0
ROM:40A43BB6         move.b    #2,8(a0)
ROM:40A43BBC         rts
ROM:40A43BBE         ; -----

```

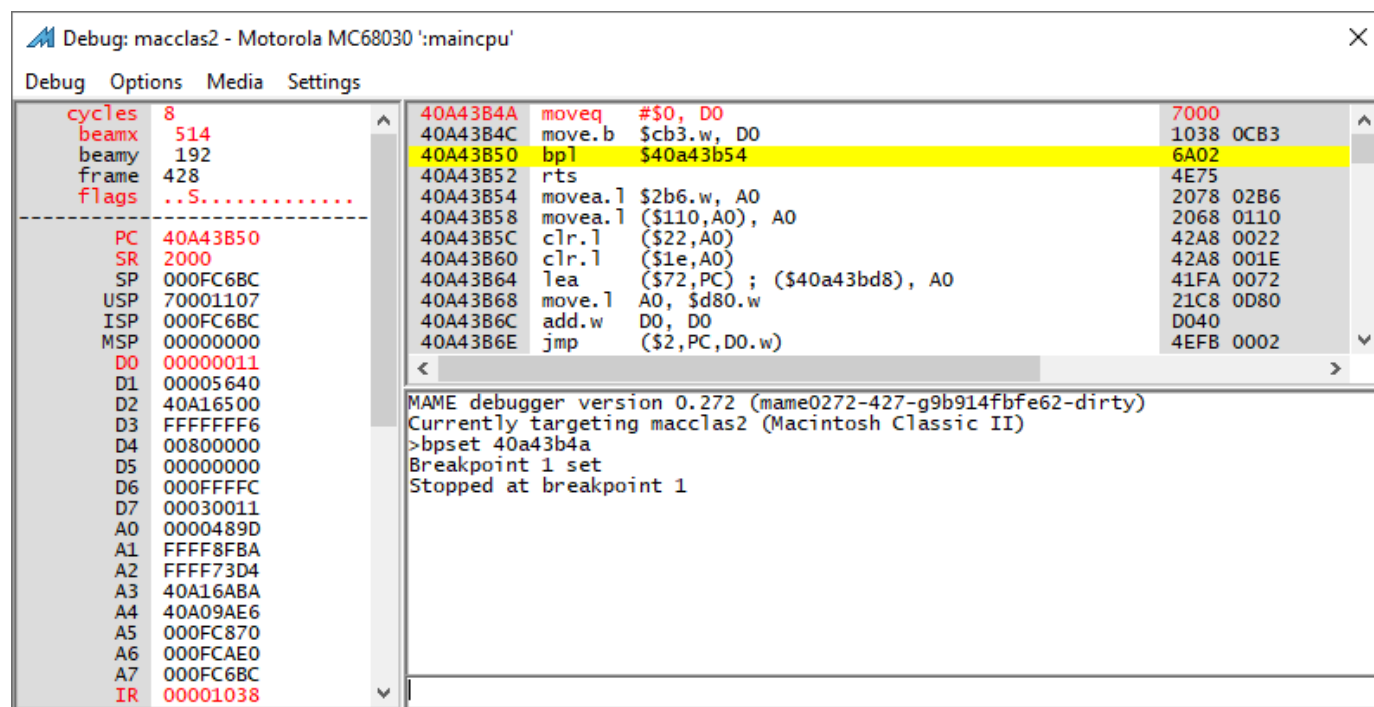

Фух! Кода тут немало. Но ничего особенно сложного в нём нет — разберём главное. По адресу 0x40A43B9C находится инструкция, которая приводит к экрану Sad Mac. Если начать с начала, код загружает байт из ОЗУ по адресу 0xCB3 в регистр D0:

```
moveq #$0, d0
```

```
move.b (byte_CB3).w, d0
```

Объяснить с  SourceCraft

Если знать, где искать, можно выяснить, что эта глобальная переменная называется VoxelFlag и хранит значение, определяющее модель машины. При пошаговом выполнении этого кода в MAME видно, что в D0 загружается 0x11, то есть 17. Для Classic II это правильное значение.



The screenshot shows the MAME debugger interface for the 'macclas2' target. The assembly window displays the following instructions:

Address	Instruction	Comment
40A43B4A	moveq	#\$0, D0
40A43B4C	move.b	\$cb3.w, D0
40A43B50	bp1	\$40a43b54
40A43B52	rts	
40A43B54	movea.l	\$2b6.w, A0
40A43B58	movea.l	(\$110, A0), A0
40A43B5C	clr.l	(\$22, A0)
40A43B60	clr.l	(\$1e, A0)
40A43B64	lea	(\$72, PC); (\$40a43bd8), A0
40A43B68	move.l	A0, \$d80.w
40A43B6C	add.w	D0, D0
40A43B6E	jmp	(\$2, PC, D0.w)

The registers window on the left shows the following values:

Register	Value
PC	40A43B50
SR	2000
SP	000FC6BC
USP	70001107
ISP	000FC6BC
MSP	00000000
D0	00000011
D1	00005640
D2	40A16500
D3	FFFFFFF6
D4	00800000
D5	00000000
D6	000FFFFC
D7	00030011
A0	0000489D
A1	FFFF8FBA
A2	FFFF73D4
A3	40A16ABA
A4	40A09AE6
A5	000FC870
A6	000FCAE0
A7	000FC6BC
IR	00001038

The status bar at the bottom indicates: 'MAME debugger version 0.272 (mame0272-427-g9b914fbfe62-dirty) Currently targeting macclas2 (Macintosh Classic II) >bpset 40a43b4a Breakpoint 1 set Stopped at breakpoint 1'.

Дальше выполняется ещё несколько операций, а затем по адресу 0x40A43B6C значение в D0 удваивается — получается 0x22. Сразу после этого оно используется как смещение в инструкции перехода. Вот как этот переход записан в IDA: такой синтаксис понятнее, чем вариант из MAME.

```
add.w d0, d0
```

```
jmp loc_40A43B72(pc, d0.w)
```

Объяснить с 

После удвоения в D0 находится 0x22, поэтому переход идёт на $0x40A43B72 + 0x22 = 0x40A43B94$. Вот как этот код выглядит в отладчике MAME, когда выполнение до него доходит:

Debug: macclas2 - Motorola MC68030 'maincpu'

Debug Options Media Settings

cycles	11	40A43B94	cas.w	D1, D0, (\$4,A4); (2+)	0CEC 08A9 0004
beamx	571	40A43B9A	move.b	D0, D4	1800
beam	192	40A43B9C	move.b	#\$90, (\$1C00,A1)	137C 0090 1C00
frame	428	40A43BA2	rts		4E75
flags	..S.....	40A43BA4	rts		4E75
		40A43BA6	movea.l	\$cec.w, A0	2078 0CEC
		40A43BAA	move.b	#\$90, (\$13,A0)	117C 0090 0013
		40A43BB0	rts		4E75
		40A43BB2	movea.l	\$cec.w, A0	2078 0CEC
		40A43BB6	move.b	#\$2, (\$8,A0)	117C 0002 0008
		40A43BBC	rts		4E75
		40A43BBE	movea.l	\$cec.w, A1	2078 0CEC

PC 40A43B94
SR 2000
SP 000FC6BC
USP 70001107
ISP 000FC6BC
MSP 00000000
D0 00000022
D1 00005640
D2 40A16500
D3 FFFFFFFF6
D4 00800000
D5 00000000
D6 000FFFFC
D7 00030011
A0 40A43BD8
A1 FFFF8FBA
A2 FFFF73D4
A3 40A16ABA
A4 40A09AE6
A5 000FC870
A6 000FCAE0
A7 000FC6BC

MAME debugger version 0.272 (mame0272-427-g9b914fbfe62-dirty)
Currently targeting macclas2 (Macintosh Classic II)
>bpset 40a43b4a
Breakpoint 1 set
Stopped at breakpoint 1

Если продолжить пошаговое выполнение, в итоге мы доберёмся до инструкции, которая вызывает Sad Mac. Посмотрим, в каком состоянии находятся регистры перед её выполнением:

Debug: macclas2 - Motorola MC68030 'maincpu'

Debug Options Media Settings

cycles	7	40A43B94	cas.w	D1, D0, (\$4,A4); (2+)	0CEC 08A9 0004
beamx	590	40A43B9A	move.b	D0, D4	1800
beam	192	40A43B9C	move.b	#\$90, (\$1C00,A1)	137C 0090 1C00
frame	428	40A43BA2	rts		4E75
flags	..S.....	40A43BA4	rts		4E75
		40A43BA6	movea.l	\$cec.w, A0	2078 0CEC
		40A43BAA	move.b	#\$90, (\$13,A0)	117C 0090 0013
		40A43BB0	rts		4E75
		40A43BB2	movea.l	\$cec.w, A0	2078 0CEC
		40A43BB6	move.b	#\$2, (\$8,A0)	117C 0002 0008
		40A43BBC	rts		4E75
		40A43BBE	movea.l	\$cec.w, A1	2078 0CEC

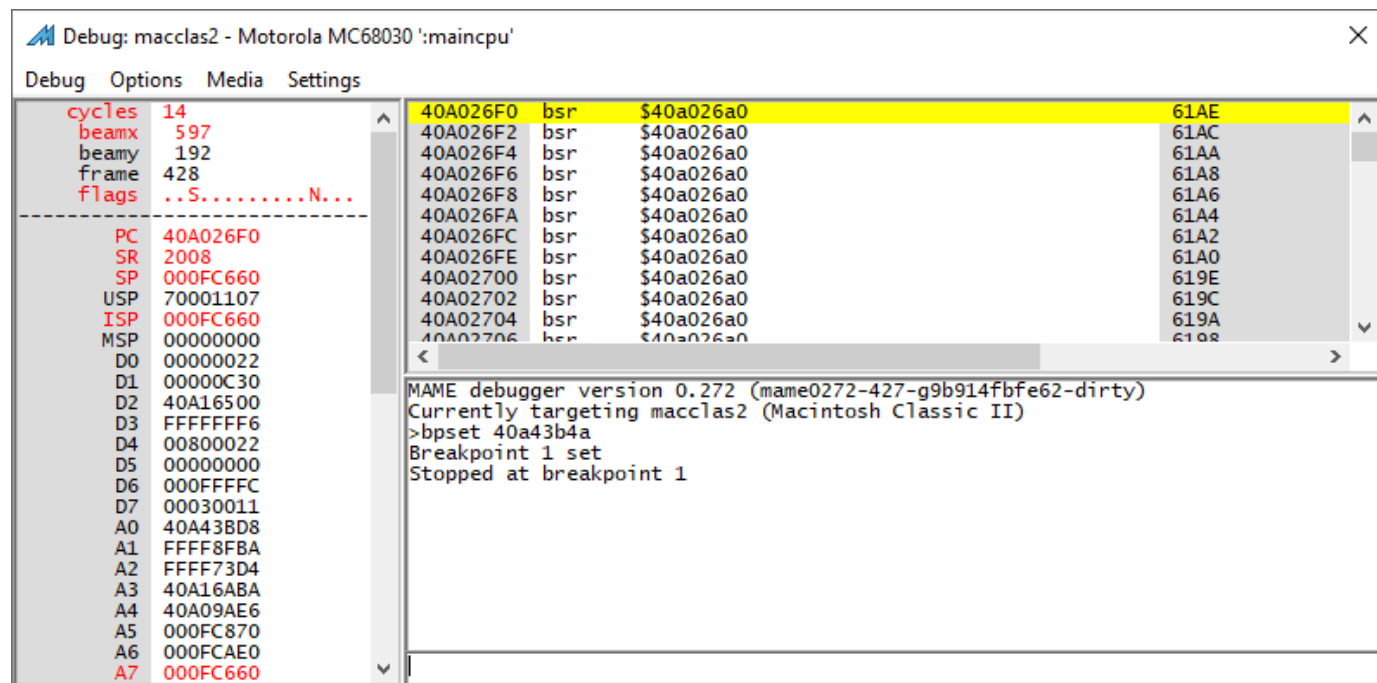
PC 40A43B9C
SR 2000
SP 000FC6BC
USP 70001107
ISP 000FC6BC
MSP 00000000
D0 00000022
D1 00000C30
D2 40A16500
D3 FFFFFFFF6
D4 00800022
D5 00000000
D6 000FFFFC
D7 00030011
A0 40A43BD8
A1 FFFF8FBA
A2 FFFF73D4
A3 40A16ABA
A4 40A09AE6
A5 000FC870
A6 000FCAE0
A7 000FC6BC

MAME debugger version 0.272 (mame0272-427-g9b914fbfe62-dirty)
Currently targeting macclas2 (Macintosh Classic II)
>bpset 40a43b4a
Breakpoint 1 set
Stopped at breakpoint 1

Хм, странно. Эта аварийная инструкция записывает значение 0x90 по адресу со смещением 0x1C00 относительно адреса из A1. В A1 находится 0xFFFF8FBA, поэтому

запись идёт по адресу $0xFFFF8FBA + 0x1C00 = 0xFFFFABBA$. Для Classic II это совершенно недопустимый адрес. Неудивительно, что появляется Sad Mac.

Как и ожидалось, стоит пошагово выполнить эту инструкцию — и вместо RTS, расположенной сразу за ней, мы попадаем в ветвь кода, которая выводит ошибку Sad Mac, по адресу $0x40A026F0$. Вот здесь всё окончательно и ломается.



Теперь я уже довольно хорошо понимал, что происходит в MAME. В A1 лежало мусорное значение, поэтому код из ПЗУ записывал данные по недопустимому адресу. FFFFABBA-дабба-ду! Я решил копнуть дальше и разобраться, как в A1 попал неверный адрес. И тут обнаружилось нечто совсем странное.

Давайте ещё раз посмотрим на один из предыдущих скриншотов — на момент после того, как по значению D0 (BoxFlag) мы перешли к нужному для Classic II фрагменту кода:

Debug: macclas2 - Motorola MC68030 ':maincpu'				X			
Debug Options Media Settings							
cycles	11	40A43B94	cas.w	D1, D0, (\$4,A4); (2+)	0CEC	08A9	0004
beamx	571	40A43B9A	move.b	D0, D4	1800		
beamy	192	40A43B9C	move.b	#90, (\$1c00,A1)	137C	0090	1C00
frame	428	40A43BA2	rts		4E75		
flags	..S.....	40A43BA4	rts		4E75		

PC	40A43B94	40A43BA6	movea.l	\$cec.w, A0	2078	0CEC	
SR	2000	40A43BAA	move.b	#90, (\$13,A0)	117C	0090	0013
SP	000FC6BC	40A43BB0	rts		4E75		
USP	70001107	40A43BB2	movea.l	\$cec.w, A0	2078	0CEC	
ISP	000FC6BC	40A43BB6	move.b	#52, (\$8,A0)	117C	0002	0008
MSP	00000000	40A43BBC	rts		4E75		
D0	00000022	40A43BBE	movea.l	\$cec.w, A1	2078	0CEC	
D1	00005640						
D2	40A16500						
D3	FFFFFFF6						
D4	00800000						
D5	00000000						
D6	000FFFFC						
D7	00030011						
A0	40A43BD8						
A1	FFFF8FBA						
A2	FFF73D4						
A3	40A16ABA						
A4	40A09AE6						
A5	000FC870						
A6	000FCAE0						
A7	000FC6BC						

MAME debugger version 0.272 (mame0272-427-g9b914fbfe62-dirty)
Currently targeting macclas2 (Macintosh Classic II)
>bpset 40a43b4a
Breakpoint 1 set
Stopped at breakpoint 1

Я ещё немного подумал и в конце концов понял, что здесь произошло нечто совершенно безумное. Мы должны были попасть в таблицу инструкций BRA.S — по одной для каждого возможного значения VoxFlag. Поэтому перед использованием D0 как смещения для перехода мы складывали его с самим собой: каждая инструкция BRA.S занимает два байта, и индекс в таблице нужно было удвоить, чтобы превратить его в байтовое смещение. Почему же мы не оказались на инструкции BRA.S? И откуда тут взялась инструкция CAS.W?

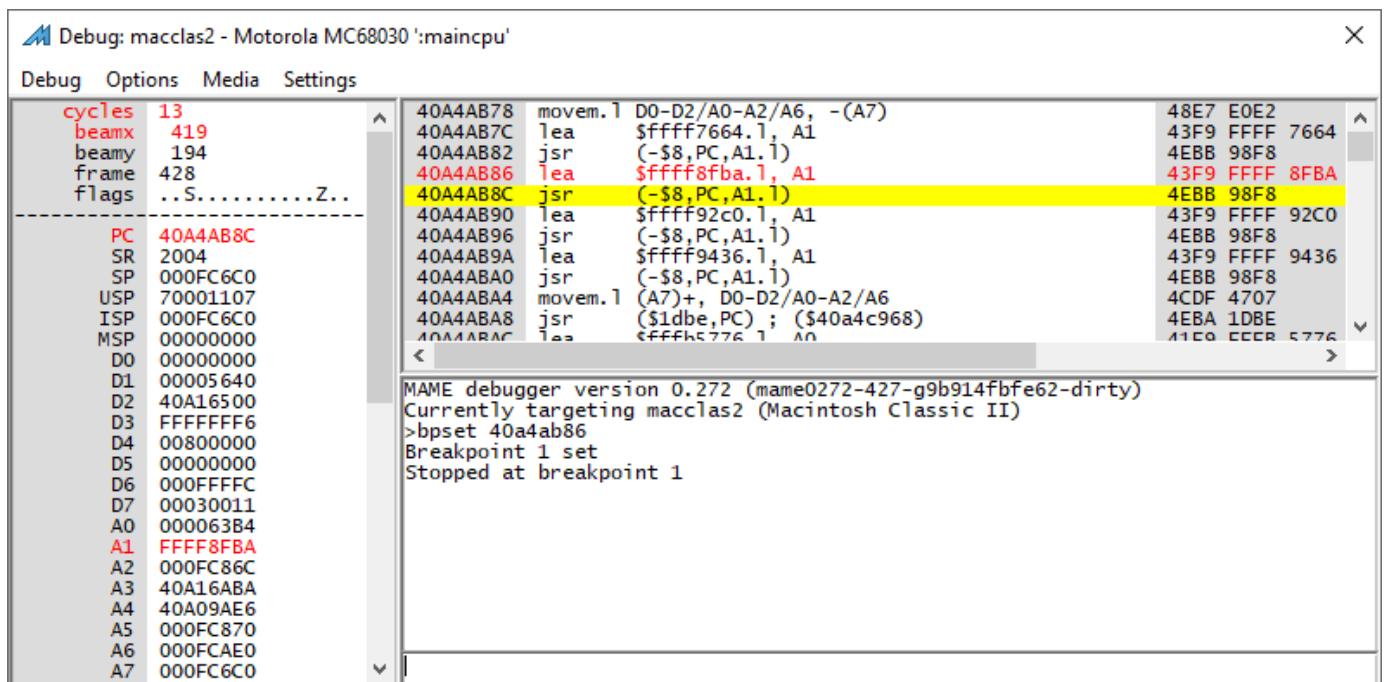
Если внимательно посмотреть на таблицу переходов под инструкцией JMP по адресу 0x40A43B6E, в ней всего 16 записей — для значений VoxFlag от 0 до 15. А у Classic II VoxFlag равен 17!

Как я уже говорил, вычисленное смещение приводит нас к адресу 0x40A43B94, который вообще не должен быть началом инструкции. Он попадает ровно в середину инструкции MOVEA.L по адресу 0x40A43B92. Именно эта инструкция должна загрузить в A1 настоящий адрес, который код затем использует для включения звукового прерывания.

Когда выполнение переходит на 0x40A43B94, процессор уже не исполняет предусмотренный код. Он сбивается с рассчитанного потока выполнения. Значение 0x0CEC должно было быть вторым словом инструкции MOVEA.L — адресом в ОЗУ, который следовало загрузить. Но вместо этого процессор воспринимает его как начало новой инструкции.

К нужной границе инструкций процессор возвращается не сразу. Сначала он выполняет загадочную инструкцию CAS (compare-and-swap, «сравнить и обменять»), затем непредусмотренную инструкцию MOVE.B D0, D4 и лишь потом доходит до настоящей MOVE.B по адресу 0x40A43B9C — той самой, которая вызывает сбой. Только в этот момент процессор вновь начинает исполнять код, который Apple действительно рассчитывала выполнить. Но в A1 уже находится недопустимый адрес, потому что код, который должен был записать туда корректное значение, был пропущен. Поэтому попытка записать данные по адресу A1 + 0x1C00 закономерно приводит к падению. Теперь всё сходится.

Если копнуть ещё раньше, окажется, что «мусорное» значение 0xFFFF8FBA загружается в A1 при первоначальном переходе к `InstallSoundIntHandler`. И, конечно, это вовсе не мусор: значение используется как смещение для инструкции перехода.



Дизассемблирование в IDA выглядит чуть понятнее. Значение 0xFFFF8FBA, загруженное в A1, показывает, сколько нужно прибавить к счётчику команд (PC), чтобы попасть из текущей точки в InstallSoundIntHandler. Если интерпретировать его как знаковое, это отрицательное число, потому что функция находится раньше по коду ПЗУ.

```
ROM:40A4AB86 loc_40A4AB86: ; CODE XREF: SoundInitPatch+141p
ROM:40A4AB86 ; DATA XREF: SoundInitPatch:loc_40A4AB86↓0
ROM:40A4AB86 lea ((InstallSoundIntHandler-loc_40A4AB86)).1,a1
ROM:40A4AB8C jsr loc_40A4AB86(pc,a1.1)
```

В итоге мне показалось, что я полностью разобрался в происходящем. Возможно, я повторяюсь, но хочу ещё раз зафиксировать главное: проблемное значение в A1 загружается как часть дальнего относительного перехода к этому участку ПЗУ, а выход за границы таблицы уводит выполнение за код, который должен был загрузить в A1 настоящий адрес периферийного устройства для настройки звуковых прерываний. В результате в A1 остаётся отрицательное смещение перехода, а не реальный адрес. Наконец, это значение используется как адрес при операции записи — и бум, получаем Sad Mac.

Если вы дочитали до этого места, у вас наверняка назрело несколько вопросов. Теперь понятно, как MAME приходит к сбою, но не почему. Почему это вообще происходило? И почему тот же сбой не проявлялся на реальном оборудовании? Очевидно, что Classic II не отзывали с рынка из-за невозможности использовать 32-битную адресацию. Такого просто не могло случиться: об этом написали бы все технические издания. Да и разработчики ПЗУ быстро заметили бы проблему во время тестирования. Слишком уж она бросается в глаза.

Так что же происходило? MAME вёл себя не так, как реальное оборудование? На железе этот код вообще не мог быть достигнут, верно? Ответы у меня есть, но сразу предупрежу:

всё оказалось куда сложнее, чем я ожидал.

Сначала я попытался понять, что делает инструкция CAS, на которую попадает выполнение после перехода за пределы таблицы. Вот её байты:

0C EC 08 A9 00 04

Я быстро заметил, что, если в IDA указать начало кода в этой точке, она вообще отказывается дизассемблировать инструкцию:

```
ROM:40A43B94      dc.w $CEC
ROM:40A43B96      dc.w $8A9
ROM:40A43B98      dc.w 4
ROM:40A43B9A ; -----
ROM:40A43B9A      move.b d0,d4
ROM:40A43B9C      move.b #$90,$1C00(a1)
ROM:40A43BA2      rts
```

Когда я попытался преобразовать данные в код, начиная с 0x40A43B94, IDA вывела:

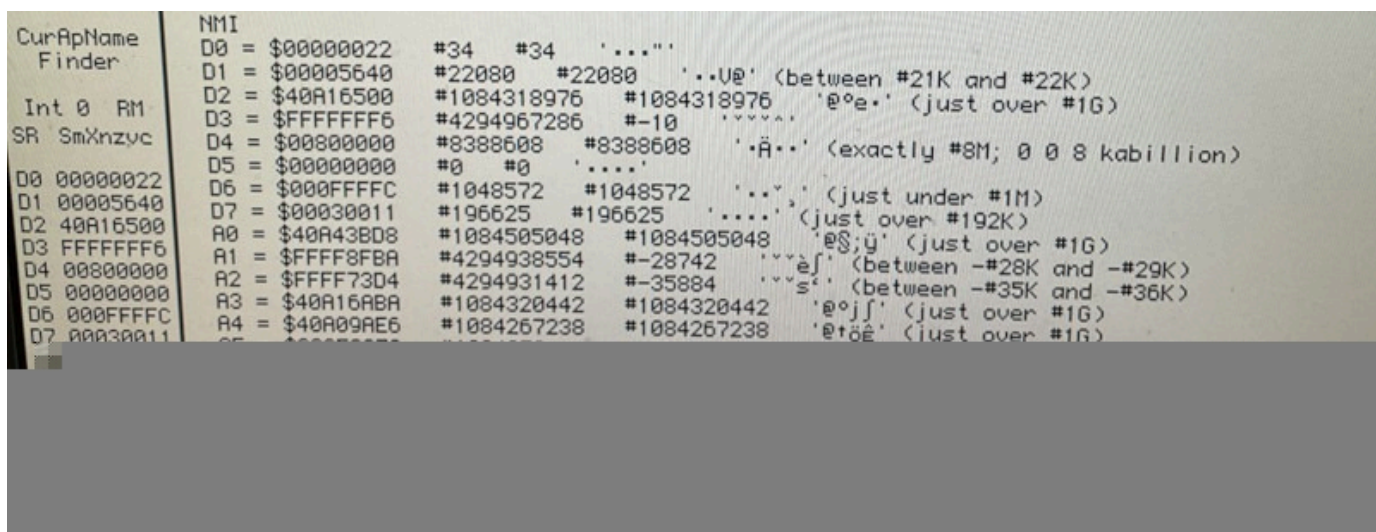
Command "MakeCode" failed

GNU objdump тоже не смог дизассемблировать эту инструкцию, а затем сразу вернулся к синхронизации с предусмотренным кодом:

```
40a43b94: 0cec                .short 0x0cec
40a43b96: 08a9 0004 1800      bclr #4,%a1@(6144)
40a43b9c: 137c 0090 1c00      moveb #-112,%a1@(7168)
40a43ba2: 4e75                rts
```

Объяснить с 

То, что два известных дизассемблера споткнулись на этой инструкции, меня заинтриговало. Я решил воспользоваться MacsBug на своём Macintosh IIci, у которого тоже установлен процессор 68030: разместить этот код в ОЗУ по случайному адресу и посмотреть, что о нём скажет MacsBug. Раз уж я зашёл так далеко, я также выставил остальные регистры в те же значения, что видел в MAME. Полностью воспроизвести состояние MAME не удалось: счётчик команд пришлось оставить указывающим на ОЗУ, а не на ПЗУ.



Любопытно: MacsBug тоже распознал инструкцию как CAS.W, но интерпретировал её немного иначе. Он показал CAS.W D1, D2, \$0004(A4) .

Разумеется, я не удержался и пошагово выполнил этот код в MacsBug, чтобы увидеть, как он поведёт себя на настоящем процессоре 68030:

Стоп... что? Если сравнить состояние регистров в левой части экрана на первом снимке и на втором, окажется, что произошло нечто крайне странное. Хотя ни MacsBug, ни MAME в своей интерпретации этой инструкции CAS вообще не упоминают A1, значение этого регистра изменилось! Сначала в нём было 0xFFFF8FBA, а затем стало 0xFC6B8. Похоже, оно превратилось в значение, похожее на содержимое A5–A7, то есть в допустимый адрес ОЗУ.

Дальнейшие эксперименты с MacsBug и разными значениями регистров показали, что новое значение A1 зависит от исходных значений A1, A7 и счётчика команд. Точно понять, что происходит, я не смог, но было очевидно: инструкция очень заметно меняет A1.

В этот момент мне показалось, что я нащупал разгадку. Эмулируемый в MAME Classic II падал, потому что A1 не менялся и в нём оставался недопустимый адрес. На реальном оборудовании эта странная инструкция, которую несколько дизассемблеров вообще не стали обрабатывать и на которую выполнение не должно было переходить, поскольку она начинается посередине другой корректной инструкции, меняла A1 на новое допустимое значение. Неужели эта безумная инструкция случайно исправляла A1 и тем самым скрывала ошибку от разработчиков ПЗУ Apple в начале 1990-х?

Примерно тогда Arbee предложил мне начать публиковать результаты исследования на форумах 68kmla и bannister.org: возможно, кто-нибудь из потрясающих людей, которые знают о наборе инструкций 68k намного больше меня, сможет подключиться к обсуждению. Я также поспрашивал в IRC-канале #mac68k сети Libera.

Общее мнение сводилось к тому, что это не корректно закодированная инструкция CAS, а MacsBug правильно определяет задействованные регистры как D1 и D2. Посмотрим, что говорится о кодировании инструкции CAS в руководстве Motorola M68000 Family Programmer's Reference Manual:

Сопоставив это с тремя словами инструкции (0x0CEC 0x08A9 0x0004) и заполнив поля, получаем следующую картину:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	SIZE		0	1	1	EFFECTIVE ADDRESS					
0	0	0	0	1	10=word		0	1	1	MODE			REGISTER		
										101=(d16,An)			100=A4		
0	0	0	0	0	0	0	Du			0	0	0	Dc		
0	0	0	0	1	0	0	010=D2			1	0	1	001=D1		

Первое слово выглядит как корректная инструкция CAS. А вот во втором есть несколько битов, равных 1, хотя формат инструкции явно требует, чтобы они были сброшены в 0. Я выделил их красным. Кроме того, поля Du и Dc совпадают с трактовкой MacsBug, а не с тем, как их интерпретирует MAME.

Третье слово, 0x0004, — это упомянутое в поле MODE значение d16: смещение \$0004 относительно A4. То есть, согласно справочному руководству Motorola, эта инструкция выглядит так:

CAS D1,D2,\$0004(A4)

...за исключением трёх битов, которые установлены в 1 там, где должны быть нули. Поэтому перед нами вообще не корректно закодированная инструкция. По крайней мере, в документации такой вариант не описан.

Кстати, похоже, отладчик MAME неправильно декодирует и обычные инструкции CAS. Если заменить последовательность на 0x0CEC 0x0081 0x0004 — то есть записать эту инструкцию корректно, без трёх лишних единичных битов, — он всё равно считает, что D0 соответствует D0, а не D2. Но это уже побочная деталь: инструкция, с которой мы разбираемся в этой истории, в любом случае полностью сломана.

Читайте ещё по теме:

Декодирование инструкций в сопроцессоре с плавающей точкой Intel 8087

Как работает движок микрокода процессора 8086

Два бита на транзистор: ПЗУ микрокода повышенной плотности в FPU-сoproцессоре Intel 8087

Инструкция CAS (compare-and-swap, «сравнить и обменять») сама по себе довольно интересна. Она позволяет выполнять разные атомарные операции без блокировок. Это одна из немногих инструкций процессоров семейства 68000, которые выполняют цикл шины «чтение—модификация—запись» (RMW).

Конкретно эта инструкция должна сравнить слово в памяти по адресу A4 + 4 со значением в D1. Если значения совпадают, содержимое D2 записывается по адресу A4 + 4. В противном случае слово из памяти по адресу A4 + 4 загружается в D1.

Судя по всему, часть этих действий она всё-таки выполняет — например, цикл «чтение—модификация—запись» для A4 + 4. Если указать в A4 недопустимый адрес, MacsBug на это жалуется. Например, на моём Mac IIci с той же тестовой конфигурацией, что и выше, я задал A4 = 0xFFFF0000 и снова выполнил некорректную инструкцию. MacsBug вывел:

Bus Error at 0004CB20

while reading word (read-modify-write) from FFFF0004 in Supervisor data space

Объяснить с 

Это однозначно показывает, что инструкция всё же выполняет RMW-цикл по адресу $A4 + 4$. Но работает она не совсем так, как должна работать CAS. Обычная CAS, разумеется, не должна менять содержимое A1.

Я провёл ещё несколько тестов, указав в A4 адрес в ОЗУ. Если записать 0xFFFF по адресу $A4 + 4$, задать $D1 = 0x1111$ и $D2 = 0x2222$, то после выполнения инструкции значение в памяти по адресу $A4 + 4$ меняется на 0x2222. Но это не имеет смысла: 0x2222 должно было быть записано только при условии, что D1 равно 0xFFFF.

Подведём промежуточный итог.

- Некорректно закодированная инструкция, на которую по ошибке переходит код ПЗУ (0x0CEC 0x08A9 0x0004), отчасти похожа на CAS D1, D2, \$0004(A4), но не совсем: некоторые биты, которые должны быть нулевыми, установлены в 1.
- На другом Mac с процессором 68030 я увидел, что эта инструкция меняет значение в регистре A1.
- Эмулятор процессора 68030 в MAME не меняет A1, поскольку инструкция недокументирована и нормальный код вообще не должен её использовать.
- Экран Sad Mac в MAME появляется через пару инструкций: в A1 остаётся недопустимый адрес, а код из ПЗУ пытается записать байт по адресу $A1 + 0x1C00$.

Я начал склоняться к мысли, которая звучала слишком безумно, чтобы оказаться правдой: в ПЗУ Classic II была ошибка с переходом за пределы таблицы, из-за которой при загрузке должен был появляться Sad Mac. Но Apple о ней не знала, потому что 68030 случайно исправлял значение A1, выполняя недокументированную инструкцию. Как доказать эту теорию?

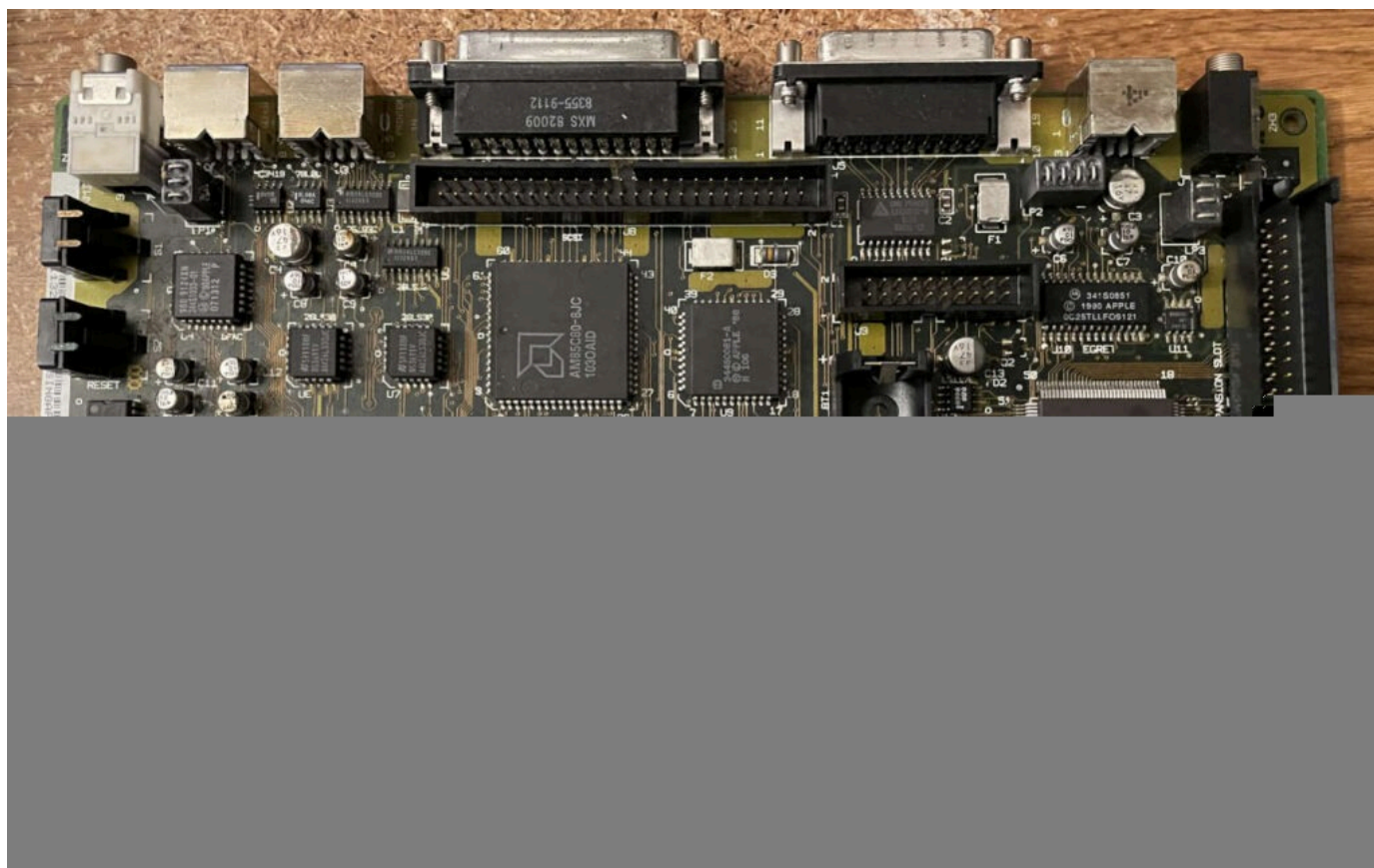
Разумеется, купить Classic II и модифицировать ПЗУ, чтобы увидеть, что именно происходит на реальном оборудовании!



Этот Classic II был выпущен в 1991 году, так что на момент написания статьи ему около 34 лет. Компьютеры такого возраста обычно требуют ремонта, если их до этого никто не восстановил. Как я упоминал в прошлой статье об ошибке с полярностью конденсаторов

в LC III, SMD-конденсаторы в старых Mac со временем начинают течь, выпуская агрессивный электролит. Этот экземпляр исключением не стал.

Я даже не пытался его включать. Сразу вскрыл корпус и снял, по всей видимости, родную литиевую батарейку Sonnenschein формата 1/2 AA с маркировкой даты «март 1991 года». К счастью, она совсем не потекла. Это ещё одна неприятность, которая часто встречается у старых Mac, десятилетиями пролежавших без дела: протёкшая батарейка может нанести им очень серьёзные повреждения.



Не хочу вдаваться в мучительные подробности ремонта и делать эту статью ещё длиннее, но, если интересно, я выложил несколько обновлений о ходе работ в своей ветке на [форуме 68kmla](#). Во время пайки новых конденсаторов я всё-таки случайно замкнул /RESET на землю, но в итоге системную плату удалось оживить. Несколько месяцев назад нечто похожее произошло с платой SE/30 у Adrian's Digital Basement. Мне, правда, повезло больше: в моём случае случайное замыкание не было связано с 12 вольтами и не спалило кучу микросхем.

В Classic II используется электронно-лучевая трубка, а она пугает меня до чёртиков. Поэтому я решил пойти другим путём и запускать системную плату отдельно, без опасных напряжений. К тому же аналоговые платы в таких компьютерах славятся тем, что ремонтировать их крайне неприятно. У этой утечка электролита была особенно сильной, так что, вероятно, она станет отдельным проектом на будущее. Суть в том, что мне предстояло много раз устанавливать и вынимать микросхемы ПЗУ, поэтому из

соображений безопасности и удобства хотелось держать системную плату на открытом месте.

Отдельное спасибо участнику форума 68kmla davewongillies: он выложил похожую конфигурацию с системной платой Classic II и добавил ссылки на Amazon для многих комплектующих. Правда, инструмент для извлечения контактов Mini-Fit Jr. с Amazon оказался полным мусором — у меня так и не получилось заставить его работать. В итоге я воспользовался скобками от степлера.

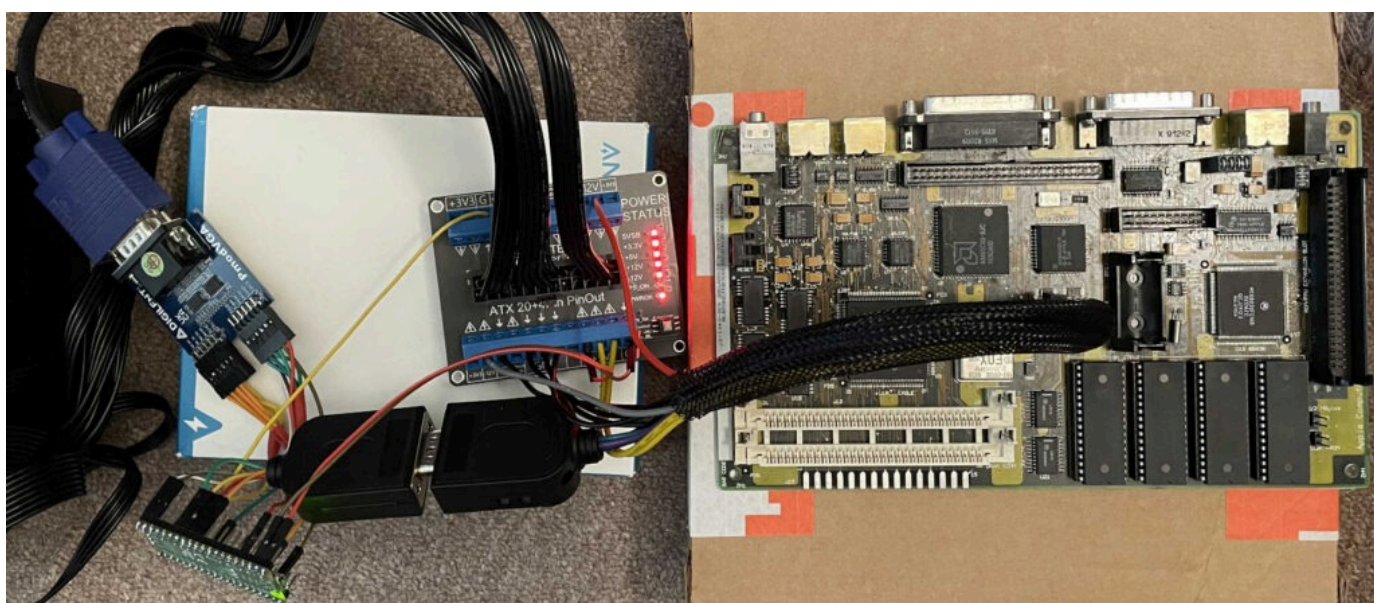


Пока я ждал RGBtoHDMI, который должен был преобразовать видеосигнал Classic II в HDMI, мне не терпелось продолжить. В конце концов я нашёл [другое решение](#) от пользователя GitHub под ником guruthree: оно предполагает использование Raspberry Pi Pico для преобразования сигнала в VGA. Я собрал несколько кабелей, подправил код, пока не добился правильных временных параметров и цветов, и получил вот такую конструкцию для перехода между видеовыходом CGA на разъёме DE-9 и VGA:



Хотя в реальном продакшене, где важна надёжность, я бы так никогда не сделал, здесь я даже не использовал преобразователи логических уровней для 5-вольтовых сигналов от Mac. Формально RP2040 «в некотором смысле» совместим с уровнями 5 В, хотя в даташите это не заявлено. В сочетании с тем, что у меня валялось несколько Pi-со, которые было не жалко угробить, этого оказалось достаточно, чтобы рискнуть.

Вот вся эта конструкция в собранном виде, вместе с блоком питания ATX. Я также заменил оригинальные микросхемы ПЗУ на имевшиеся у меня программируемые EEPROM SST29EE010. Любопытно, что заводские микросхемы Apple на самом деле были EPROM с УФ-стиранием, так что при наличии УФ-стиратора я мог бы использовать и оригинальные чипы.



С этой конфигурацией мне удалось загрузиться с SCSI-накопителя и захватить видеосигнал одним из моих многочисленных устройств видеозахвата:



Кстати, кажется, что как минимум с одним столбцом пикселей в левой части экрана что-то не так. Думаю, проблема в доработке кода на RP2040, которую я внёс, чтобы он правильно синхронизировался с видеосигналом Classic II. Наверняка это можно исправить, если ещё немного поэкспериментировать. У RGBtoHDMI, который приехал через несколько дней, такой проблемы нет.

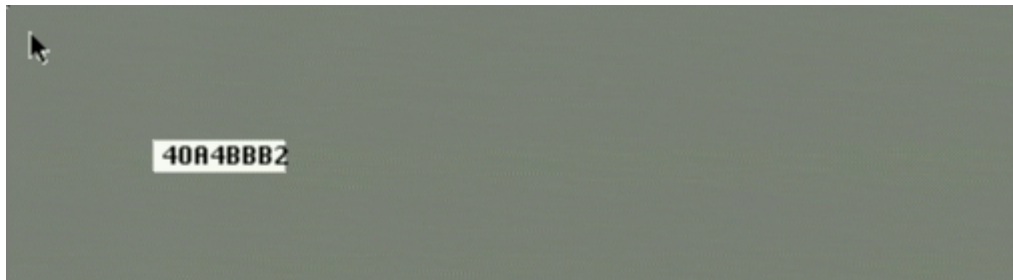
В общем, я понял, что двигаюсь в правильном направлении. Я на скорую руку написал ассемблерный код для 68030, который выводит значение A1 на экран, нашёл в ПЗУ свободное место для него и подготовил три модифицированных образа ПЗУ для проверки на Classic II:

1. **Образ ПЗУ 1.** Заменить инструкцию по адресу 0x40A43B9C, из-за которой появляется Sad Mac — то есть MOVE.B, — переходом к моему специальному коду, выводящему A1 на экран. Так можно увидеть, какое значение A1 имеет на реальном оборудовании в этот момент. Заодно это подтвердит, что данный участок кода вообще выполняется на реальной машине.
2. **Образ ПЗУ 2.** Заменить инструкцию по адресу 0x40A43B94 — CAS — тем же переходом к специальному коду. Это должно подтвердить, что переход за пределы таблицы

действительно происходит, а также показать значение A1 непосредственно перед ним.

3. **Образ ПЗУ 3.** Заменить инструкцию по адресу 0x40A43B94 — CAS — на NOP. В идеале это должно в точности воспроизвести то, что я наблюдал в MAME, и доказать, что некорректная инструкция CAS критически важна для загрузки Classic II.

Разберём результаты по очереди. Вот что появилось на экране при запуске образа ПЗУ 1:



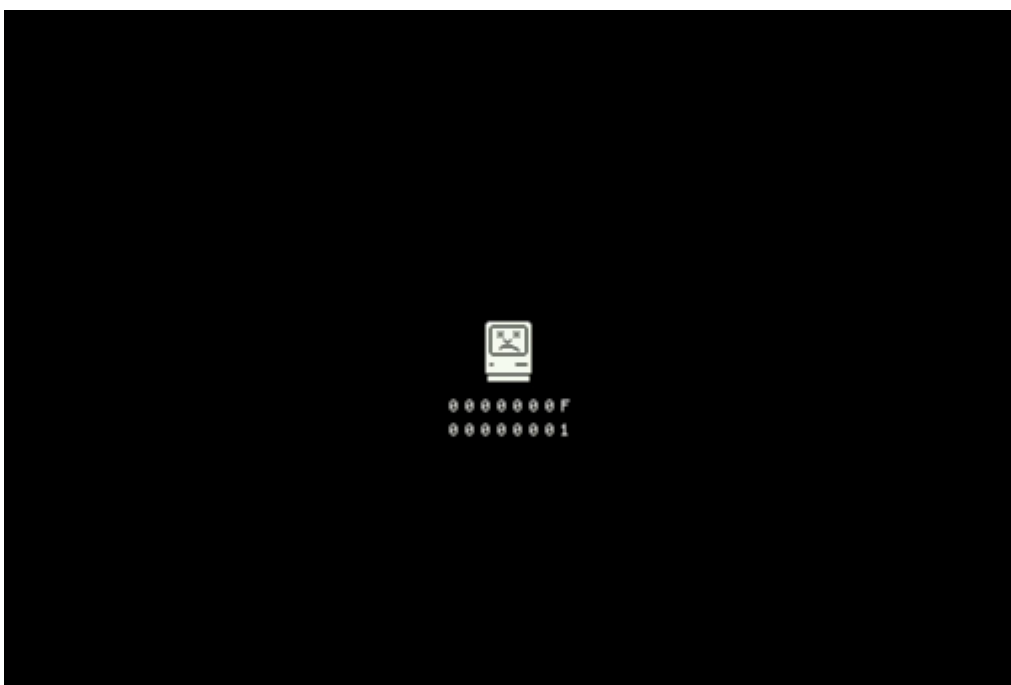
Это подтвердило, что исследуемый участок кода действительно выполняется на реальном оборудовании. Кроме того, оказалось, что к моменту выполнения инструкции, которая приводит к сбою в MAME, в A1 находится очень интересное значение. 0x40A4BBB2 — это адрес в ПЗУ, так что записывать по нему в обычной ситуации не стали бы. Но попытка записи не вызывает ошибки шины.

А вот что получилось при запуске образа ПЗУ 2:



Значение A1 оказалось тем же, что и в MAME! Это доказало сразу две вещи. Во-первых, переход за границы таблицы точно происходил. Иначе мой код, выводящий A1 на экран, вообще не выполнялся бы: JMP на него находится вне нормального потока инструкций, точно так же, как случайная CAS. Во-вторых, вместе с первым тестом это подтвердило мою гипотезу: на реальном оборудовании CAS действительно исправляет A1.

Наконец, запуск образа ПЗУ 3, где CAS вообще исключена из картины, дал мне последнее нужное доказательство:



Sad Mac — точно такой же, как в MAME при 32-битной адресации. Во время этого теста я также выяснил, что на реальном оборудовании такой же Sad Mac возникает и в 24-

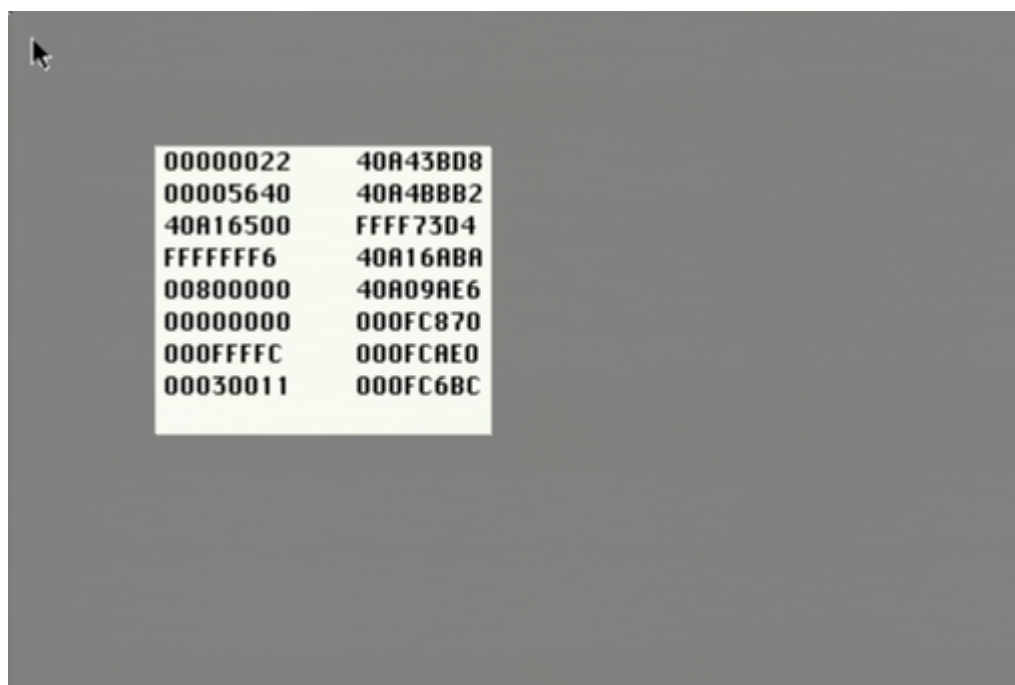
битном режиме. Получается, MAME в 24-битном режиме терпимее к этой некорректной записи, чем реальная машина.

Эти результаты подтолкнули меня подготовить ещё пару модифицированных образов ПЗУ и запустить их на реальном оборудовании, чтобы снять значения всех регистров данных и адресных регистров процессора непосредственно перед CAS и сразу после неё. Регистры данных показаны в левом столбце, адресные — в правом. До:



00000022	40A43BD8
00005640	FFFF8FBA
40A16500	FFFF73D4
FFFFFFFF6	40A16ABA
00800000	40A09AE6
00000000	000FC870
000FFFFC	000FCAE0
00030011	000FC6BC

И после:



00000022	40A43BD8
00005640	40A4BBB2
40A16500	FFFF73D4
FFFFFFFF6	40A16ABA
00800000	40A09AE6
00000000	000FC870
000FFFFC	000FCAE0
00030011	000FC6BC

Да! Всё осталось прежним, кроме A1, который каким-то чудесным образом превратился из FFFF8FBA в 40A4BBB2. За это явно отвечает загадочная инструкция.

Ещё одна приятная часть этого теста — удалось убедиться, что до некорректной инструкции реальное оборудование ведёт себя в точности так же, как MAME. Всё состояние регистров на снимке «до» полностью совпадает с тем, что MAME показывает при загрузке в 32-битном режиме перед некорректной CAS. Смотрите сами:

D0	00000022	00000022	40A43BD8
D1	00005640	00005640	FFFF8FBA
D2	40A16500	40A16500	FFFF73D4
D3	FFFFFFF6	FFFFFFF6	40A16ABA
D4	00800000	00800000	40A09AE6
D5	00000000	00000000	000FC870
D6	000FFFFC	000FFFFC	000FCAE0
D7	00030011	00030011	000FC6BC
A0	40A43BD8		
A1	FFFF8FBA		
A2	FFFF73D4		
A3	40A16ABA		
A4	40A09AE6		
A5	000FC870		
A6	000FCAE0		
A7	000FC6BC		

Если после всего этого у вас закипел мозг, то я вас прекрасно понимаю. Но давайте попробуем собрать всё воедино и разобраться, что это значит.

Я обнаружил недокументированную инструкцию MC68030, которая выполняет цикл шины «чтение-модификация-запись» и одновременно меняет значение регистра A1.

Эта новооткрытая инструкция оказалась случайной скрепой, на которой держится Classic II. Без изменения A1 этой инструкцией Classic II не может загрузиться. Я уверен, что это ошибка, а не намеренное решение. Причём вполне понятная ошибка. Если бы этот коварный 68030 не скрывал баг от разработчиков ПЗУ Apple, они наверняка заметили бы его до выхода Classic II в продажу.

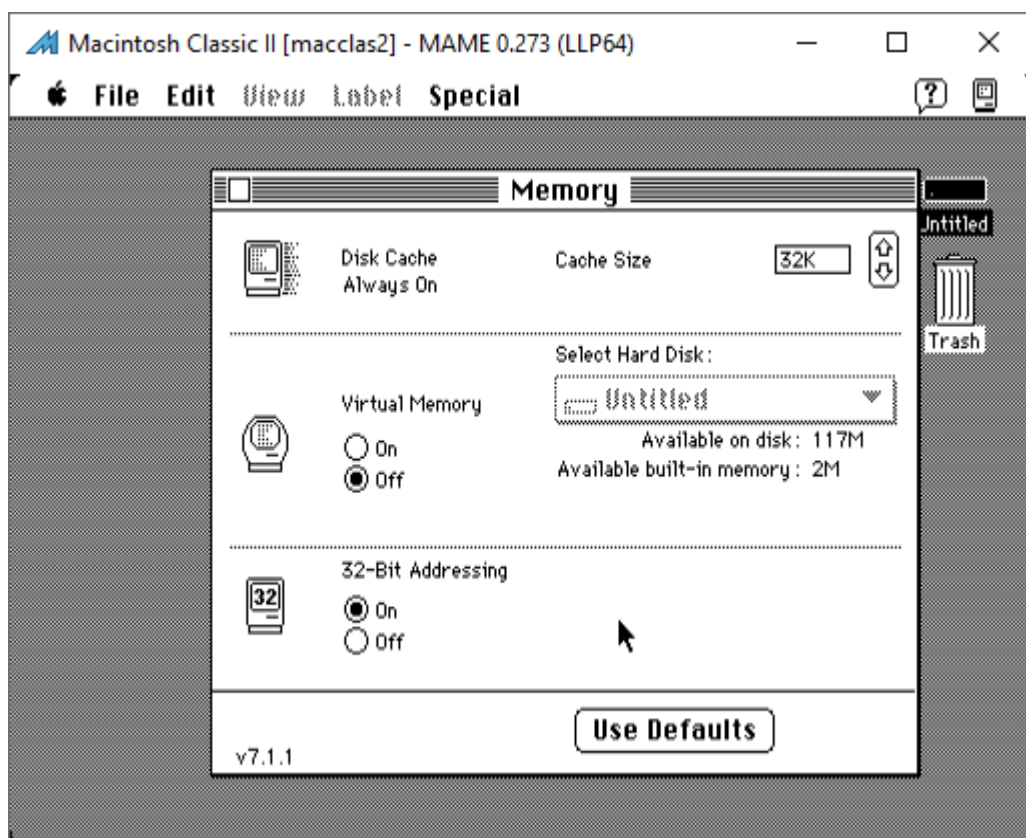
Я поискал дальше и нашёл тот же фрагмент кода в более новом ПЗУ Macintosh IIvx. В этой версии размер таблицы переходов наконец увеличили. Я подтвердил, что ветка для Classic II в том коде вообще ничего не делает: она сразу переходит к инструкции RTS. Интересно, чесал ли затылок разработчик Apple, работавший над этим участком ПЗУ IIvx, когда добавлял записи для новых моделей, включая Classic II, хотя ПЗУ Classic II уже было готово и поставлялось с компьютерами. Кто знает. Я не уверен, как Apple тогда управляла всем набором вариантов ПЗУ.

Из этого открытия следует, что сейчас, скорее всего, не существует на сто процентов точного эмулятора или реплики Motorola MC68030. Возможно, это единственный известный случай, когда такая разница вообще важна. То есть я мог бы написать небольшой фрагмент кода, который определяет, запущен он на физическом 68030 или в эмуляторе: достаточно выполнить эту инструкцию и посмотреть, какое значение получится в A1.

Что может сделать MAME, чтобы обойти проблему и позволить Classic II загружаться? Мы пока точно не знаем, что именно делает эта инструкция. По результатам ограниченной серии тестов я предполагаю, что итоговое значение A1 зависит от исходного A1, значения A7 и счётчика команд. Но полной уверенности нет. Возможно, кто-то напишет программу, которая переберёт множество комбинаций значений регистров и содержимого памяти, а затем попытается восстановить точную логику инструкции, чтобы корректно её эмулировать.

Пока никто не решил, что стоит разобраться в этом до конца, в MAME просто патчат эту ошибку в ПЗУ, чтобы Classic II мог загрузиться. Как заметил Arbee, мы немного опоздали с просьбой к Motorola, Freescale или NXP выпустить errata. Разве что эту статью случайно увидит кто-то, работавший над 68030, и сможет пролить свет на происходящее...

Вот скриншот MAME с патчем Arbee: теперь он успешно эмулирует Classic II с включённой 32-битной адресацией. Ура!



Какой вывод можно сделать из этой истории? Наверное, такой: эмуляторы способны рассказать нам о железе то, что мы бы никогда не додумались исследовать сами. Готов поспорить, что этот баг в ПЗУ остался бы неизвестным навсегда, если бы MAME не эмулировал Classic II — не самую заметную модель на фоне более популярных компактных Mac вроде SE/30 и Color Classic.

Эта история также показывает, что ошибки могут годами скрываться там, где, казалось бы, всё уже давно вылизано. И ещё это хороший пример того, что не каждая ошибка

действительно критична. Этот баг как раз из таких: машина нормально работала, и никто ничего не заметил.

Ах да, изначально я вообще полез в это расследование из-за сочетания Command + Power. В MAME оно не работает. Теперь, когда у меня есть настоящий Classic II, я подтвердил, что на реальном оборудовании это сочетание действительно работает. Правда, только при установленном MacsBug — вероятно, из-за того, что Egret по умолчанию его отключает, о чём я писал выше. Но в любом случае при установленном MacsBug оно должно работать и в MAME. Похоже, у меня появился ещё один баг MAME, которым стоит заняться!



Продолжить разбираться в низкоуровневой разработке можно на бесплатных открытых уроках OTUS. На встречах получится посмотреть на подход преподавателей, задать вопросы и точно закрыть пробелы в теме.

- **23 июля, 20:00.** «Фаззинг и реверс: как понять, что делает программа, найти в ней ошибки». Записаться
- **27 июля, 20:00.** «Разработка Embedded-устройств для IoT». Записаться

Теги: Macintosh Classic II, Motorola MC68030, apple, реверс-инжиниринг, эмуляция, архитектура процессора, ПЗУ, винтажные компьютеры

Хабы: Блог компании OTUS, Реверс-инжиниринг, Старое железо, Компьютерное железо

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Электронная почта

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



256K+

Охват за 30 дней

OTUS

[Сайт](#) [ВКонтакте](#) [Telegram](#)



128K+

173

Охват за 30 дней

Карма

Ксения Мосеенкова @kmoseenk

Переводчик, редактор

Подписаться



Комментировать

Публикации

ЛУЧШИЕ ЗА СУТКИ

ПОХОЖИЕ



GlinkinIvan

22 часа назад

Хакер в магазине: изучаем поверхность атаки типичного супермаркета



9 мин



14K



+56



46



39



medicalofficer1111
22 часа назад

Пить коллаген — это развод?



7 мин



9.3K



+37



39



22



arturdumchev
23 часа назад

Калвиаутра путает калвиши: один байт изменил всё



Средний



8 мин



11K

Тutorial



+35



7



7



PatientZero
3 часа назад

Теория мёртвой экономики



21 мин



6.4K

Перевод



+24



15



20



nastyakopi
21 час назад

ИИ-роутер, open-source RAG-платформа, ClickHouse-as-a-service и другие апдейты в продуктах Selectel в июне



7 мин



11K

Дайджест



+23



7



3



inetstar
20 часов назад

Полный геном за \$200 и его анализ в домашних условиях: Just-DNA-Lite, ИИ и пересборка генома. Часть 3



Средний



29 мин



10K

Tutorial

 +20

 17

 4



SLY_G

19 часов назад

Сколько элементарных частиц существует на самом деле?



Простой



9 мин



10K

Перевод

 +19

 16

 12



xraizor

3 часа назад

Усталость от напарника-машины. Изнанка работы с ИИ-код-агентами



Средний



11 мин



5.3K

Мнение

 +16

 7

 8



AWE64

15 часов назад

Мне надоело писать один и тот же код. Поэтому я сделал Featuregen



Средний



6 мин



13K

Кейс

 +16

 14

 4



inspired_xy

17 часов назад

Как мы съездили в Японию и запустили Пополаму — сервис, который знает кто кому должен



Простой



4 мин



11K

Кейс

Из песочницы

 +16

 13

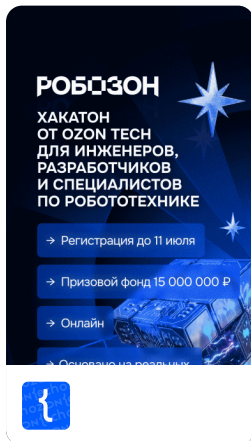
 4

Меряем глубину проникновения ИИ в разработку

Турбо

Показать еще

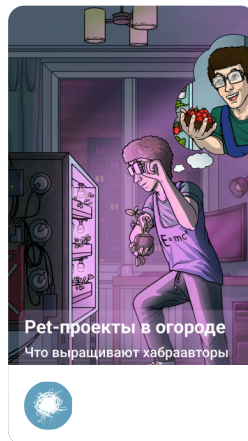
ИСТОРИИ



Унеси с Робозона свои миллионы



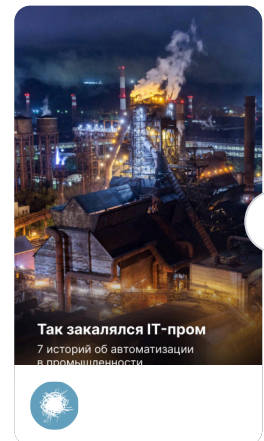
Пришёл, смастерил, рассказал — сезон DIY в разгаре



От лунного овоща до сверхострого чили

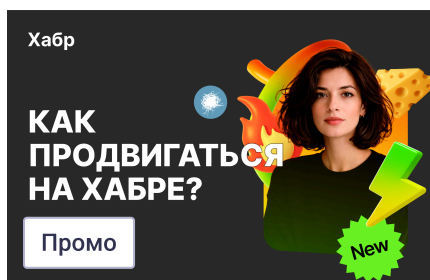


Из центра Млечного Пути

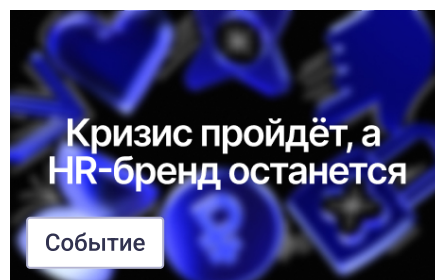


Только IT-пром, только хардкор

МИНУТОЧКУ ВНИМАНИЯ



Экскурсия по Хабру для маркетингологов



Бесплатный вебинар о том, как строить HR-бренд в 2026 году



DPL в Overlay-слое: общение виртуальных машин



 [Настройка языка](#)

[Техническая поддержка](#)

© 2006–2026, Habr